UXL SKILLS EVALUATOR

oneDAL Skill

API paths, computation modes, tables, parity, and conversion cost

September 2026 · 5 of 6 evaluation tasks implemented

Unified Acceleration Foundation

What the skill tells an agent today

- **Select sklearn extension, native C++, or DAAL maintenance** — The answer depends on change tolerance, queue control, language, and legacy context.
- **Define the analytics contract** — Preprocessing, split, parameters, seeds, outputs, metric, and tolerance.
- **Map tables deliberately** — Use rectangular fixtures to expose orientation bugs that square data can hide.
- **Validate each computation mode** — Chunk coverage and finalization for online; partitions, rank participation, and merge for distributed.
- **Benchmark only after parity** — Include conversion, communication, synchronization, and environment.

Evaluation inventory: 5 implemented, 1 honest gap

- **Implemented** — sklearn-or-native KMeans; table orientation regression; batch/online/distributed choice; conversion-cost benchmark; Extra Trees random split.
- **Incident evidence** — The Extra Trees task is sourced from a maintainer incident.
- **Planned** — An authentic unavailable-GPU-path case on a declared target.
- **What this proves** — The suite checks path selection, semantics, and evidence discipline — not every algorithm/device combination.

How we chose the content

- **Official sources** — Project docs, examples, release state, algorithm/API guidance, and ownership files.
- **High-cost failure boundaries** — Wrong API path, transposed tables, unsuitable computation mode, quality drift, and hidden conversion cost.
- **Project-level truth** — Exact algorithm/interface/version/device availability must be checked rather than copied into a static matrix.
- **Measured restraint** — A tiny-corpus agentic prototype was kept as a no-go result, not promoted as a speedup.

What project ownership would look like

- **Review scope and migration nuance** — Especially sklearn/native boundaries, oneAPI versus DAAL, and mode availability.
- **Validate parity language** — Which outputs require exact, tolerance-based, or metric-based comparison?
- **Nominate real failures** — Prefer authentic algorithm, table, or device incidents over synthetic coverage.
- **UXL keeps the machinery** — Common evaluator policy, tasks, dashboards, hosted CI, and specialized runner contracts.

A focused 30-minute maintainer review

- **1. Confirm the path-selection guidance** — When should an agent recommend sklearn acceleration, native C++, or legacy DAAL maintenance?
- **2. Check mode and table gotchas** — Correct one overstatement and add one missing real-world failure.
- **3. Judge the six tasks** — Keep, rewrite, or remove; nominate a credible unavailable-GPU case.
- **4. Name an owner** — One person or team for release-time or quarterly review.
- **Outcome** — Agents arrive with the workload, API path, data contract, parity method, and benchmark scope already clear.